

St. Francis Institute of Technology, Mumbai-400 103
Department of Information Technology

A.Y. 2025-2026

Class: BE-ITA/B, Semester: VII

Subject: Healthcare Lab

Experiment 3

- 1. Aim:** To read and analyze X-ray image for medical diagnosis.
- 2. Objectives:** Students should be able to develop healthcare applications using appropriate computational tools.
- 3. Prerequisite:** Python basics
- 4. Pre-Experiment Exercise:**

Theory:

X-Ray imaging is one of the oldest and still one of the most prevalent methods of medical imaging. X-ray is the life-saving technology that was actually invented by accident. X-ray images are grayscale images, that is, the images have some pixels which are dark and some that are bright. In medical imaging terms, these are images that have values ranging from 0 to 255, where 0 corresponds to the completely dark pixels, and 255 corresponds to the completely white pixels.

Different values on the X-ray image correlate to different areas of density:

- Dark — Locations on the body which are filled with Air are going to appear black.
- Dark Grey — Subcutaneous tissues or fat
- Light Grey — Soft tissues like the heart and blood vessels
- Off White — Bones such as the ribs
- Bright White — Presence of metallic objects such as pacemakers or defibrillators

The way physicians interpret an image is by looking at the borders between the different densities. As in Figure 1, the ribs appear off-white because they are dense tissues, but since the lungs are filled with air, the lungs appear dark. Similarly below the lung is the hemidiaphragm, which is again a soft tissue and hence appears light grey. This helps us give a clear understanding of the location and extent of the lungs.

So, if two objects with different densities are present close to each other they can be demarcated in an X-ray image.

Now if something were to happen in the lungs, such as Pneumonia, then, the air dense lungs will change into water-dense lungs. This will cause the demarcation lines to fade since the pixel densities will start closing in on the grayscale bar.

For taking a chest X-ray, normally the patient is asked to stand, and the X-rays are shot from either front to back (Anterior-Posterior) or from back to front (Posterior-Anterior).

Various Anatomical structures which can be differentiated using Chest X-rays:

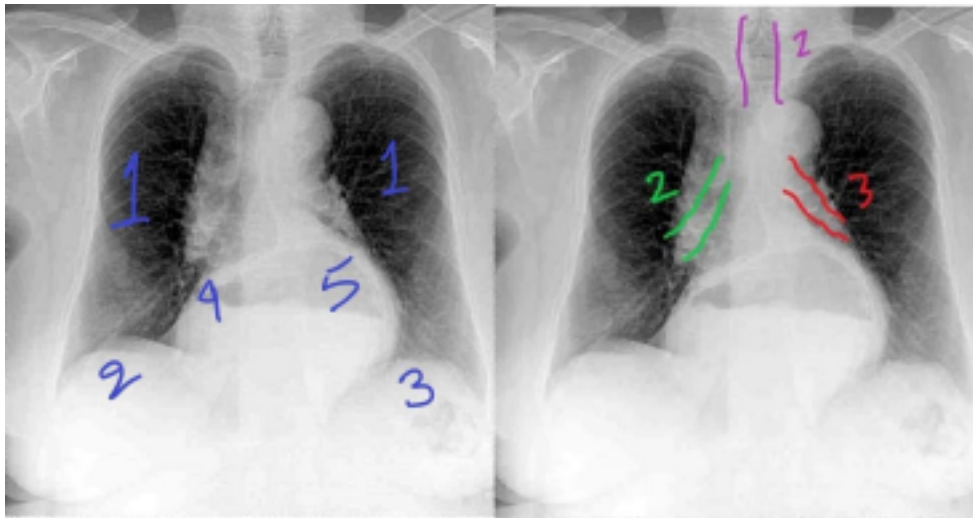


Figure 1.

Chest X-Ray — 1) Lungs, 2) Right Hemidiaphragm, 3) Left Hemidiaphragm, 4) Right Atrium, 5) Left Atrium (By Diego Grez — Radiografía_pulmones_Francisca_Lorca.jpg, CC BY-SA

3.0, <https://commons.wikimedia.org/w/index.php?curid=10302947>. Editing by Author)

Figure 2. Anatomy Airways — 1) Trachea, 2) Right Main bronchus, 3) Left main Bronchus (By Diego Grez — Radiografía_pulmones_Francisca_Lorca.jpg, CC BY-SA

3.0, <https://commons.wikimedia.org/w/index.php?curid=10302947>. Edited by Author)

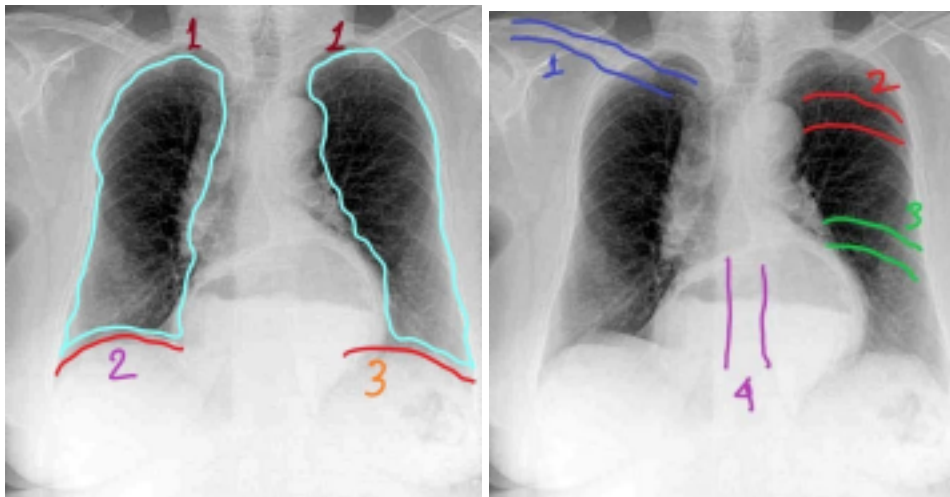


Figure 3. Anatomy Diaphragm and Pleura — 1) Pleura, 2) Right Hemidiaphragm, 3) Left Hemidiaphragm (By Diego Grez — Radiografía_pulmones_Francisca_Lorca.jpg, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=10302947>. Edited by Author)

The point of the above images is to show how difficult it is for someone who is not a trained doctor, to interpret the different anatomies from a Chest X-ray image, let alone any abnormalities.

That is exactly where Deep learning comes in handy. With deep learning, even people who have no idea about the different abnormalities or anatomies of the body can come up with decent results on predicting various anomalies.

interpret X-ray images, let's look at a few techniques to view and analyze the images, their histograms, and a technique to add images and labels together, using Python programming.

6. Laboratory Exercise

Procedure

- Download a healthcare dataset with X-ray images.
- Interpret X-ray images
- Analyze the images,
- Create their histograms
- Apply a technique to add images and labels together, using Python programming.

Post-Experiments Exercise:**A. Extended Theory:**

- a. Apply X-ray image analysis for COVID diagnosis using X-ray dataset

B. Conclusion:

1. Write what was performed in the program (s) .
2. What is the significance of program and what Objective is achieved?

C. References:

- [1]<https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia/data>.
- [2]<https://towardsdatascience.com/deep-learning-in-healthcare-x-ray-imaging-part-3-analyzing-images-using-python-915a98fbf14c>

Laboratory Exercise**Procedure**

 `pip install torch torchvision matplotlib pillow numpy`

```

Requirement already satisfied: torch in /usr/local/lib/python3.11/dist-packages (2.6.0+cu124)
Requirement already satisfied: torchvision in /usr/local/lib/python3.11/dist-packages (0.21.0+cu124)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (3.10.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.11/dist-packages (11.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (2.0.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch) (3.18.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/dist-packages (from torch) (4.14.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.11/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch) (2025.3.0)
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)

```

```

import os
import matplotlib.pyplot as plt
from PIL import Image
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms

# ----- STEP 1: DATASET DEFINITION -----
class XRayDataset(Dataset):
    def __init__(self, root, transform=None, max_per_class=None):
        self.samples = []
        self.transform = transform
        for label, cls in enumerate(["NORMAL", "PNEUMONIA"]):
            cls_path = os.path.join(root, cls)
            files = [f for f in os.listdir(cls_path) if f.endswith((".jpeg", ".jpg", ".png"))]
            files = sorted(files)[:max_per_class] if max_per_class else files
            for f in files:
                self.samples.append((os.path.join(cls_path, f), label))

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = Image.open(path).convert("L")
        if self.transform:
            img = self.transform(img)
        return img, label

```

```

# ----- STEP 2: SIMPLE CNN MODEL -----
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
        self.fc1 = nn.Linear(16 * 32 * 32, 64)
        self.fc2 = nn.Linear(64, 2)

    def forward(self, x):
        x = F.relu(F.max_pool2d(self.conv1(x), 2))
        x = F.relu(F.max_pool2d(self.conv2(x), 2))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)

# ----- STEP 3: TRAINING FUNCTION -----
def train_model():
    print("📁 Preparing data...")
    train_transform = transforms.Compose([
        transforms.Resize((128, 128)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
    ])

    # Load and limit samples
    full_ds = XRayDataset("chest_xray/train", transform=train_transform)
    normal_samples = [s for s in full_ds.samples if s[1] == 0][:50]
    pneumonia_samples = [s for s in full_ds.samples if s[1] == 1][:50]
    combined_samples = normal_samples + pneumonia_samples

    # Custom dataset from combined samples
    class SubsetDataset(Dataset):
        def __init__(self, samples, transform):
            self.samples = samples
            self.transform = transform

        def __len__(self):
            return len(self.samples)

        def __getitem__(self, idx):
            path, label = self.samples[idx]
            img = Image.open(path).convert("L")
            if self.transform:
                img = self.transform(img)
            return img, label

    train_ds = SubsetDataset(combined_samples, train_transform)
    train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

```

```

model = SimpleCNN()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

model.train()
for epoch in range(5):
    total_loss = 0
    for xb, yb in train_loader:
        optimizer.zero_grad()
        out = model(xb)
        loss = loss_fn(out, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"✅ Epoch {epoch+1}: Loss = {total_loss / len(train_loader):.4f}")

return model, train_transform

print(f"✅ Displayed {shown_normal} NORMAL and {shown_pneumonia} PNEUMONIA examples.")

# ----- RUN PIPELINE -----
if __name__ == "__main__":
    model, tf = train_model()

```

 Preparing data...

- ✅ Epoch 1: Loss = 0.7006
- ✅ Epoch 2: Loss = 0.6947
- ✅ Epoch 3: Loss = 0.6982
- ✅ Epoch 4: Loss = 0.6938
- ✅ Epoch 5: Loss = 0.6938

Output

```

# ----- STEP 5: INFERENCE -----
def predict(model, transform):
    model.eval()
    test_ds = XRayDataset("chest_xray/test", transform=transform)

    shown_normal = 0
    shown_pneumonia = 0
    max_to_show = 3 # Show 3 examples per class
    shown = 0

    for i in range(len(test_ds)):
        if shown_normal >= max_to_show and shown_pneumonia >= max_to_show:
            break

        img, label = test_ds[i]
        x = img.unsqueeze(0)

        with torch.no_grad():
            pred = model(x).argmax(1).item()

        is_normal = (label == 0)
        if is_normal and shown_normal < max_to_show:
            shown_normal += 1
        elif not is_normal and shown_pneumonia < max_to_show:
            shown_pneumonia += 1
        else:
            continue # Skip if we've already shown enough from this class

        title = f"True: {'PNEUMONIA' if label else 'NORMAL'} | Predicted: {'PNEUMONIA' if pred else 'NORMAL'}"
        img_np = img.squeeze().numpy()
        plt.imshow(img_np, cmap='gray')
        plt.title(title)
        plt.axis('off')
        plt.show()
        shown += 1

    print(f"✅ Displayed {shown_normal} NORMAL and {shown_pneumonia} PNEUMONIA examples.")
if __name__ == "__main__":
    predict(model, tf)

```


True: NORMAL Predicted: PNEUMONIA  True: NORMAL Predicted: PNEUMONIA 	True: NORMAL Predicted: PNEUMONIA  True: PNEUMONIA Predicted: PNEUMONIA 
True: PNEUMONIA Predicted: PNEUMONIA 	True: PNEUMONIA Predicted: PNEUMONIA  ✅ Displayed 3 NORMAL and 3 PNEUMONIA examples.

	image_file	corona_result	path
0	Normal/images/Normal-859.png	Negative	/kaggle/input/covid19-radiography-database/COV...
1	Normal/images/Normal-158.png	Negative	/kaggle/input/covid19-radiography-database/COV...
2	Normal/images/Normal-10121.png	Negative	/kaggle/input/covid19-radiography-database/COV...
3	Normal/images/Normal-1811.png	Negative	/kaggle/input/covid19-radiography-database/COV...
4	Normal/images/Normal-97.png	Negative	/kaggle/input/covid19-radiography-database/COV...

```

from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Train-test split (80-20)
train_df, test_df = train_test_split(data, test_size=0.2, random_state=42, stratify=data['corona_result'])

# Image preprocessing
datagen = ImageDataGenerator(rescale=1./255)

train_gen = datagen.flow_from_dataframe(
    dataframe=train_df,
    x_col='path',
    y_col='corona_result',
    target_size=(128, 128),
    class_mode='binary',
    batch_size=128
)

test_gen = datagen.flow_from_dataframe(
    dataframe=test_df,
    x_col='path',
    y_col='corona_result',
    target_size=(128, 128),
    class_mode='binary',
    batch_size=128
)

```

Found 11046 validated image filenames belonging to 2 classes.
Found 2762 validated image filenames belonging to 2 classes.

Splitting into train and test dataset

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

# Build a simple CNN
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(train_gen, validation_data=test_gen, epochs=5, batch_size=128)

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `in
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `P
self._warn_if_super_not_called()
Epoch 1/5
87/87 ━━━━━━━━━━━ 118s 1s/step - accuracy: 0.7610 - loss: 0.5706 - val_accuracy: 0.8860 - val_loss: 0.2842
Epoch 2/5
87/87 ━━━━━━━━━━━ 37s 424ms/step - accuracy: 0.8881 - loss: 0.2659 - val_accuracy: 0.9080 - val_loss: 0.2320
Epoch 3/5
87/87 ━━━━━━━━━━━ 37s 425ms/step - accuracy: 0.9084 - loss: 0.2293 - val_accuracy: 0.9175 - val_loss: 0.2178
Epoch 4/5
87/87 ━━━━━━━━━━━ 37s 423ms/step - accuracy: 0.9309 - loss: 0.1735 - val_accuracy: 0.9272 - val_loss: 0.1813
Epoch 5/5
87/87 ━━━━━━━━━━━ 38s 433ms/step - accuracy: 0.9498 - loss: 0.1363 - val_accuracy: 0.9388 - val_loss: 0.1649

```

Training 5 epochs


```

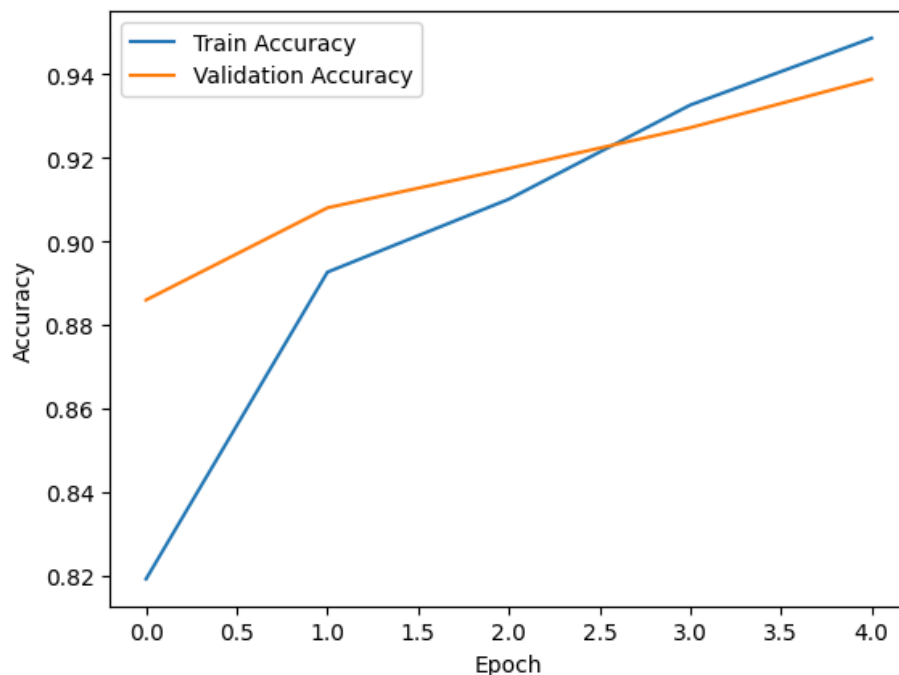
) import matplotlib.pyplot as plt

# Evaluate accuracy
loss, acc = model.evaluate(test_gen)
print(f"Test Accuracy: {acc*100:.2f}%")

# Plot accuracy curve
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

```

22/22 ————— 7s 316ms/step - accuracy: 0.9371 - loss: 0.1635
 Test Accuracy: 93.88%



93.33 percent test accuracy

```

) import numpy as np
import matplotlib.pyplot as plt

# Get class labels from generator
class_labels = list(test_gen.class_indices.keys())

# Get one batch of images and predictions
x_batch, y_batch = next(test_gen)
y_pred = model.predict(x_batch)
y_pred_labels = (y_pred > 0.5).astype(int).flatten()

# Plot 9 random images with actual and predicted labels
plt.figure(figsize=(10,10))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_batch[i])
    actual = "Positive" if y_batch[i]==1 else "Negative"
    predicted = "Positive" if y_pred_labels[i]==1 else "Negative"
    color = 'green' if actual == predicted else 'red'
    plt.title(f"Actual: {actual}\nPred: {predicted}", color=color)
    plt.axis('off')
plt.tight_layout()
plt.show()

```

Actual: Positive
Pred: Positive



Actual: Negative
Pred: Negative



Actual: Negative
Pred: Negative



Actual: Positive
Pred: Positive



Actual: Negative
Pred: Negative



Actual: Negative
Pred: Negative



Actual: Negative
Pred: Negative



Actual: Positive
Pred: Positive



Actual: Positive
Pred: Positive



Output on kaggle dataset